CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter describes the methods used to construct and evaluate the project's autonomous trajectory prediction and state-estimation prototype. The implementation combines a browser-based Three.js flight simulation, a prediction-only error-state Kalman filter (ESKF), a frozen Gated Recurrent Unit (GRU) model that estimates velocity residuals, and a separate camera-based PULP-DroNet obstacle-perception branch. The chapter focuses on components that are present in the current repository and records what can be traced to source code, configuration files, the supplied progress report, or the supplied dashboard captures.

The work has two related but distinct evidence streams. Offline model development used timestamped recorded-flight data, flight-disjoint splits, residual labels, and a 100 Hz sequence pipeline as documented in the research-progress materials and frozen GRU integration bundle. The browser demonstration instead generates simulated flight motion and synthetic inertial inputs, forms ESKF features online, and calls a local Python model service. A result from the browser therefore demonstrates software integration and behavior of that simulation setup; it is not a replay of the recorded training flights or evidence of physical flight performance.

The method is organized from system boundaries and data preparation through model design, training evidence, runtime integration, execution, and evaluation. Where the full original training source, raw corpus, or evaluator implementation is not present locally, the chapter labels the result as report-derived or screenshot-derived rather than independently reproducible. This distinction is necessary for a technically defensible account of the work.

Standalone numbering No thesis preliminary pages are included in this chapter file. Page numbering therefore begins at 1 for the chapter itself; the university's separate preliminary-page sequence can be applied

when the chapter is inserted into the complete thesis.

3.2 Overall System Architecture

The implemented application is a local, multi-process prototype. Its primary user interface and flight simulation run in a web browser using JavaScript, Three.js, and Vite. The simulation advances its navigation and inertial-data pipeline at a fixed 100 Hz step, independently of the browser's rendering cadence. Two Python FastAPI services can be started alongside it: one loads the frozen GRU V3 checkpoint for motion-residual inference, and the other loads PULP-DroNet V3 for image-based steering and collision inference. The GRU branch is the learned state-correction path; DroNet is an adjacent perception input to obstacle avoidance and does not estimate the GRU velocity residual.

The offline research pipeline is shown separately in Figure 3.1 because training and runtime are not the same process. Recorded flights are prepared and split before sequence construction; prediction-only ESKF quantities and aligned ground-truth velocity define features and residual targets; normalized windows are used to train or select a frozen checkpoint [8], [9]. During browser execution, the frontend sends only the recent 101-by-24 feature window to the local GRU endpoint. Ground truth and flight identifiers are not sent to the runtime model.

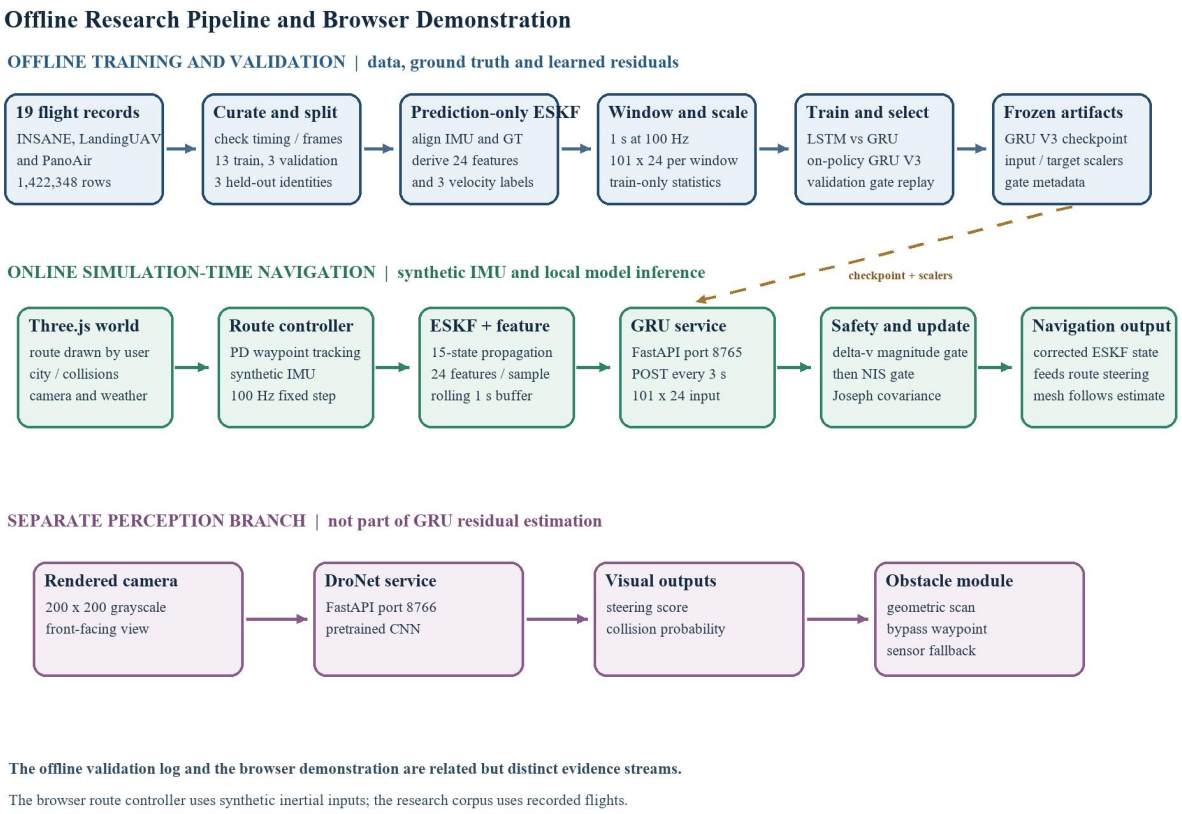


Figure 3.1. Offline GRU data and model preparation alongside the browser-time navigation and separate perception branches. The diagram distinguishes recorded-flight evidence from simulated runtime inputs.

Within the browser, the route-planning interface supplies waypoints to a guidance controller. The simulated vehicle and sensor pipeline provide specific force and angular-rate samples to the ESKF. The filter propagates its nominal navigation state and covariance; each fixed step also contributes a feature row to a rolling buffer. GRUBridge resamples the latest one-second interval to exactly 101 samples, transmits it to the local service at the demonstration cadence, and receives a three-component Δv estimate. The bridge subjects the result to the runtime magnitude/jump check and then requests an ESKF velocity measurement update, whose implementation applies a three-degree-of-freedom Normalized Innovation Squared (NIS) test. Accepted corrections influence the estimated velocity/state used by downstream route guidance; rejected or unavailable responses do not constitute an accepted GRU update.

A separate camera sensor renders the simulated scene to a grayscale image. The obstacle-avoidance client sends this image to the DroNet service and combines its outputs with geometric obstacle scanning and bypass logic. The repository also provides a sensor-side or heuristic fallback path. Accordingly, the architecture contains two learned services, but the supplied trajectory screenshots identify only the GRU/ESKF estimates and do not isolate DroNet's causal contribution.

Table 3.1. Main components and their implemented responsibilities

Component	Implementation	Input → output / responsibility
Browser simulation	Three.js / JavaScript; Vite	World, vehicle, route visualization and fixed-step simulation state.
Route controller	src/navigation/RouteController.js	Drawn route and estimated state → waypoint guidance and synthetic IMU measurements.
ESKF	src/navigation/ESKF.js	IMU propagation and covariance → nominal position, velocity, attitude and uncertainty; gated velocity update.
GRU bridge and service	GRUBridge.js; FastAPI on 127.0.0.1:8765	101×24 feature window → learned Δv, then guarded ESKF pseudo-velocity update.
DroNet perception	ObstacleAvoidanceSystem.js; FastAPI on 127.0.0.1:8766	200×200 grayscale image → steering and collision probability for obstacle logic.
Offline preparation	Recorded-flight pipeline described by report/configuration	Flight records → flight-disjoint splits, ESKF feature windows, residual labels and frozen model bundle.

The proposed Model Predictive Control (MPC/NMPC) component mentioned in planning material is not implemented in the current project tree. The browser uses a waypoint route controller rather than an MPC solver. Panda3D-related material is outside this chapter's scope, as requested; the methodology below concerns the current Three.js browser application and its connected inference services.

3.3 Development Environment and Tools

The repository is a JavaScript/Node browser application with Python inference services. Tool roles and versions are reported only to the level supported by the checked-in manifests. The package manifest specifies Three.js ^0.170.0 and Vite ^6.0.0; the README states Node.js v18 or newer. The root requirements file declares minimum versions for the principal Python runtime packages [10]. A precise operating-system, CPU, GPU, CUDA, and training-machine

specification is not established by the files examined and is therefore not asserted here.

Table 3.2. Verified software tools and their roles

Tool / package	Declared version or interface	Role in this project
JavaScript / Node.js	README: Node.js ≥18; package.json uses ES modules	Runs the frontend development/build tooling.
Three.js	^0.170.0	Renders the city, vehicle, camera, and simulation visuals.
Vite	^6.0.0	Serves the browser application and builds the frontend bundle.
Python	README: 3.10+; requirements are in root and service folders	Runs the GRU and DroNet API processes.
PyTorch	Root requirement: ≥2.2; GRU runtime uses checkpoint loading	Loads and executes the frozen GRU and CNN models.
FastAPI / Uvicorn	Root requirements: FastAPI ≥0.115, Uvicorn ≥0.30	Exposes local health and inference routes over HTTP.
NumPy / Pillow	Root requirements: NumPy ≥1.26, Pillow ≥10	Array validation, normalization, and image decoding/preparation.

The configuration and integration files are part of the model contract. In particular, the frozen GRU folder contains the runtime loader, checkpoint, train-only normalization CSV files, input/window configuration, selected safety-gate metadata, FastAPI bridge, and integration contract. The repository does not contain a complete, executable copy of the historical recorded-data training pipeline and raw dataset used for the reported experiments. Consequently, installing the present runtime dependencies can support inference but does not, by itself, reproduce model training.

3.4 Dataset Preparation and Data Integration

The supplied research-progress report describes a curated corpus assembled from three real-flight sources. It reports 1,422,348 synchronized rows from 19 flights: six INSANE flights (218,204 rows), seven LandingUAV flights (237,413 rows), and six PanoAir flights (966,731 rows). The source proportions are respectively 15.34%, 16.69%, and 67.96%. These counts are report-derived: the raw archives and full extraction/training scripts needed to recalculate them are not present in the current workspace. They should be cited as the documented corpus size, not as an independently recomputed project result.

Table 3.3. Dataset sources and flight-level partition recorded in the research-progress materials

Source	Flights	Rows	Reported share	Role / caveat
INSANE	6	218,204	15.34%	Real-flight inertial/navigation records; source channel harmonization described in report.
LandingUAV	7	237,413	16.69%	Real-flight records; environmental/location tags are categorical descriptors.
PanoAir	6	966,731	67.96%	Largest source contribution; high influence on pooled row count.
Total	19	1,422,348	100.00%	Synchronized rows across source flights, not an equal-flight corpus.

The recorded-data workflow, as described by the progress report and GRU bundle, starts by sorting observations by time and checking duplicates, discontinuities, and source-specific orientation conventions [8], [9]. Inertial measurements and ground-truth (GT) state are aligned offline. Where required, signals are reconstructed from recorded PX4 channels, and timestamp-aware interpolation places them on a common 100 Hz grid. Interpolation is constrained to valid intervals rather than extrapolating through missing or discontinuous regions. The two rotation columns used by the model are re-orthogonalized with Gram–Schmidt after interpolation [4]. These preparation steps are documented in the supplied materials; they cannot be replayed here against the absent raw files.

Flight identity is the split unit. The reported partition assigns 13 entire flights to training, three to validation, and three to held-out testing, before overlapping sequence windows are generated. Its row totals are 923,976 training, 240,977 validation, and 257,395 test. This order is important: neighboring windows share most of their samples, so splitting after window generation could place near-duplicate time segments in different partitions. The split is therefore more conservative than random window-level shuffling, although the report's later Safe V3 results are validation results and not a pristine external test.

Table 3.4. Flight-disjoint split and model-ready window counts documented by the report

Partition / stage	Flights or regimes	Rows / windows	Interpretation
Training corpus	13 flights	923,976 rows	Flight identities assigned before overlapping windows.
Validation corpus	3 flights	240,977 rows	Used for selection and safety-gate calibration.
Held-out test corpus	3 flights	257,395 rows	Available in early evaluation; not a pristine final Safe V3 test.
Base model-ready windows	Train / validation / test	2,771 / 868 / 677	One-second windows from valid segments; 4,316 total.
V3 training mixture	Open-loop / V1 closed-loop / V2 on-policy	2,771 each; 8,313 total	Three regimes balanced by window count; not additional raw rows.

The feature vector contains 24 channels and the target is a three-dimensional velocity residual. Ground-truth velocity is used to construct the offline target but is excluded from the runtime input. The feature groups and order are shown in Table 3.5; the exact feature identifiers are also encoded in model_runtime.py and the GRU configuration files.

Table 3.5. Exact GRU input order and semantic groups

Indices	Features	Meaning / units
1–3	gyro gx, gy, gz	Angular rate, rad/s.
4–6	accel ax, ay, az	Specific force, m/s².
7–9	ESKF velocity vx, vy, vz	Prediction-only nominal velocity, m/s.
10–15	Rotation-6D: r11, r21, r31, r12, r22, r32	First two columns of body-to-world rotation; six scalars.
16–18	Position covariance σpx, σpy, σpz	Per-axis position uncertainty, m.
19–21	Velocity covariance σvx, σvy, σvz	Per-axis velocity uncertainty, m/s.
22–24	Attitude uncertainty σθx, σθy, σθz	Per-axis attitude uncertainty, rad.

$$\Delta v(t) = v_{GT}(t) - v_{ESKF,pred}(t) \quad (3.1)$$

Here v_GT is the aligned recorded ground-truth velocity and v_ESKF,pred is the velocity from the prediction-only estimator. The network learns this residual at the endpoint of each window; it does not predict absolute position.

The selected context is one second sampled at 100 Hz, represented by 101 samples including both endpoints. A native evaluation correction stride of 0.5 s is documented, while browser demonstration requests are made every 3.0 s; the latter is a demonstration setting, not the training/evaluation stride. Windows are not allowed to cross a

recorded segment boundary. The base model-ready dataset contains 2,771 training, 868 validation, and 677 test windows. Input and target means and standard deviations are estimated from training data only, stored in separate CSV files, and applied to validation/test and runtime inference. The V3 training set later combines equal numbers of windows from open-loop, V1 closed-loop, and V2 on-policy regimes; that mixture does not mean additional sensor rows were fabricated.

Environmental labels such as clear, rain, wind, or night are categorical tags in the report, not calibrated numerical weather measurements. The corpus materials do not establish synchronized wind speed, rainfall intensity, or temperature for each window. Any weather-related interpretation must therefore remain descriptive and must not be written as an experimentally isolated meteorological effect.

3.5 Machine-Learning Model Architecture

3.5.1 GRU V3 residual estimator

The primary learned model is Project Ghost GRU V3, a PyTorch sequence-to-vector regressor. Its input has shape 101×24 for one inference item. A two-layer GRU has 128 hidden units per layer and inter-layer dropout of 0.20. The final recurrent output (the hidden representation at the last time step) is normalized by LayerNorm(128), then passed through a 128-to-64 fully connected layer, ReLU, dropout of 0.10, and a 64-to-3 output layer. The checkpoint/runtime bundle reports 166,915 trainable parameters. The output is de-normalized into $\Delta v=[\Delta vx, \Delta vy, \Delta vz]$ in m/s using train-only target statistics.

Table 3.6. GRU V3 network stages

Stage	Configuration	Purpose
Sequence input	101 time steps × 24 channels	One second of aligned IMU and prediction-only ESKF context.
Recurrent encoder	Two-layer GRU; hidden size 128; dropout 0.20	Encode temporal motion and estimator-error cues.
Last-step representation	Sequence output at t=101	Maps the window to an endpoint residual estimate.
Normalization	LayerNorm(128)	Normalize the final recurrent representation.
Regression head	Linear 128→64; ReLU; dropout 0.10; Linear 64→3	Produce normalized three-axis velocity residual.

Stage	Configuration	Purpose
Output transform	Target train mean/std	Return Δv in physical units (m/s).

A GRU is a gated recurrent architecture designed to retain useful temporal information while controlling information flow through update and reset gates [2]. Long Short-Term Memory (LSTM) is an alternative gated recurrent architecture [3]. The project materials record an earlier window-level LSTM-versus-GRU comparison and a later on-policy GRU V3 pipeline. These are different experiments: the early comparison informs model choice, while the final frozen V3 checkpoint was selected after additional regime-mixed training and validation [8], [9]. The selected network predicts a velocity residual; it does not directly generate a path, identify obstacles, or replace the state estimator.

3.5.2 Prediction-only ESKF and learned pseudo-measurement

The ESKF is the classical estimator that propagates the nominal state and covariance between learned corrections. Its error state has 15 components, grouped as position error δp, velocity error δv, attitude error δθ, accelerometer-bias error δba, and gyroscope-bias error δbg, each with three axes. The nominal attitude is represented by a quaternion; the implementation uses gravity [0, −9.80665, 0] in its simulation frame. IMU-driven propagation updates position, velocity, attitude, and covariance at the fixed 0.01 s step. The filter and rotation/error-state formulation follow standard inertial error-state estimation concepts [1].

$$x_e = [\delta p^T, \delta v^T, \delta\theta^T, \delta b_a{}^T, \delta b_g^T]^T \in \mathbb{R}^{15} \quad (3.2)$$

At inference, the GRU residual is added to the prediction-only ESKF velocity to form a pseudo-velocity observation. The update innovation is compared with its predicted innovation covariance, and a three-degree-of-freedom NIS statistic is tested against the configured 99% chi-square threshold (approximately 11.3449). The ESKF uses a Joseph-form covariance update for numerical stability when a measurement is accepted. A failed gate means the learned pseudo-measurement is rejected; it is not a conventional independent velocity sensor measurement.

$$v_pseudo = v_ESKF + \Delta v_GRU \quad (3.3)$$

$$NIS = y^T S^{-1} y \quad (3.4)$$

The innovation y is the difference between the pseudo-velocity observation and the predicted velocity; S is its innovation covariance. The implementation rejects an update when the three-component NIS exceeds its threshold.

3.5.3 PULP-DroNet V3 perception model

The repository contains a separate PULP-DroNet V3 CNN used by the obstacle-perception branch. Its image input is one grayscale channel of size 200×200, normalized to [0,1]. A 5×5 stride-2 convolution maps 1 channel to 32, followed by batch normalization, ReLU6, and 2×2 max pooling. Three residual blocks reduce the spatial dimensions while increasing channels 32→32→64→128; each block contains two 3×3 convolutions with batch normalization and ReLU6 and a 1×1 stride shortcut when required. The flattened 128×7×7 representation feeds a two-output linear layer. Steering is a raw scalar; a sigmoid maps the collision output to a probability. This follows the compact visual-navigation role associated with DroNet [7], while the checked-in implementation and bundled checkpoint define the actual project interface [10].

The current service loads the bundled ResBlock-1.0 weights, reports its device and weight-file status through a health endpoint, and accepts base64 images, flat pixel arrays, or image grids. The JavaScript obstacle module combines model outputs with geometric obstacle checks and route-bypass logic. No project-specific CNN training script or labeled image dataset is present locally; the chapter therefore describes inference integration and the bundled pretrained model, not training performed as part of this project.

3.5.4 Model boundary and non-implemented components

The GRU and DroNet have different input spaces, targets, and downstream effects. GRU V3 uses a short inertial/ESKF sequence to estimate Δv; DroNet uses an image to estimate steering and collision likelihood. The ESKF is a filter rather than a learned neural model. No YOLO detector, onboard flight controller, GPS receiver integration, or

MPC optimization solver is established by the inspected runtime. They are not included as implemented methodology.

3.6 Model Training Methodology

The training procedure below is the procedure documented in the supplied research-progress report, model configurations, and frozen integration bundle [8], [9]. It should not be confused with a fresh reproduction from this workspace: the complete raw data archive and original training scripts/notebooks are not available in the current repository. The package contains a runtime checkpoint and normalization files, so inference architecture and input/output contracts can be inspected, but several training hyperparameters cannot be verified from executable source here.

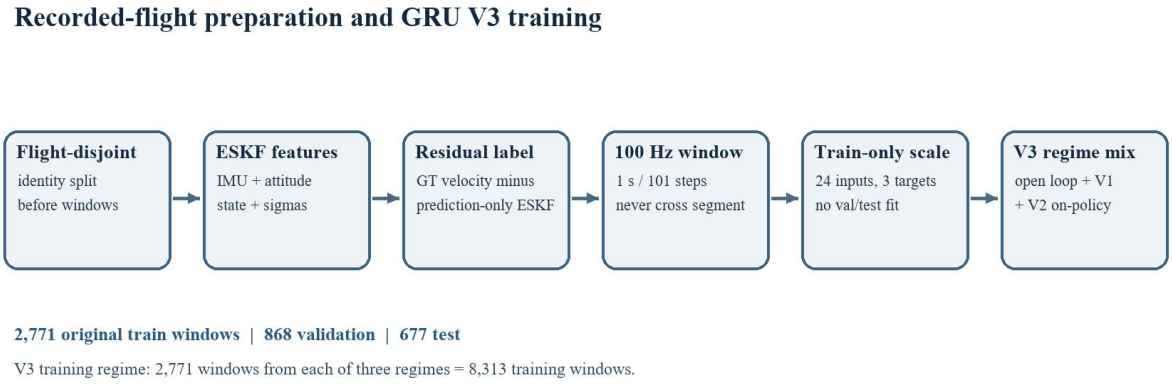


Figure 3.2. Documented recorded-flight preparation and GRU V3 training stages. Counts and training regimes are taken from the supplied report and model bundle.

The documented sequence is as follows. First, 19 recorded flights are curated and assigned by flight identity to training, validation, and held-out test partitions. Second, aligned IMU channels and prediction-only ESKF outputs are converted into the exact 24-channel feature order; ground-truth velocity supplies the residual target. Third, valid segments are interpolated onto the common 100 Hz grid, checked for discontinuities, and divided into one-second 101-sample endpoint-labeled windows without crossing segment boundaries. Fourth, normalization statistics are fitted on training data alone. Finally, a later V3 training set balances 2,771 windows each from open-loop, V1

closed-loop, and V2 on-policy regimes, and a checkpoint is selected using validation procedures documented in the report.

The progress report records an early LSTM-versus-GRU comparison: 219,651 LSTM parameters and 166,915 GRU parameters; best-epoch macro validation velocity-residual RMSE values of 71.631 m/s for LSTM and 41.952 m/s for GRU; and logged training durations of 9.00 s and 9.30 s respectively. These are early window-level comparison figures reported by the supplied materials, not the final Safe V3 recursive-navigation result. The reported GRU advantage in that comparison was its lower macro validation residual RMSE with fewer parameters. The very large residual values also underline that this comparison should not be mistaken for operational flight accuracy.

Table 3.7. Early LSTM–GRU comparison reported in the progress document

Model	Parameters	Best validation macro residual RMSE	Reported training time	Interpretation
LSTM	219,651	71.631 m/s	9.00 s	Earlier window-level comparison.
GRU	166,915	41.952 m/s	9.30 s	Lower reported macro residual error and fewer parameters.

For the later V3 experiment, the report states that training ended at epoch 44 and checkpoint epoch 30 was selected. It reports one-step validation velocity-residual RMSE of 2.239752 m/s (macro flight) and 2.325972 m/s (global), with MAE 1.747488 m/s under that evaluation distribution. The corresponding frozen runtime records its architecture and a stored validation metric. The selected safety policy was tuned on validation data: an out-of-distribution (OOD) distance threshold from train-only summary statistics and ESKF innovation/jump gates were evaluated without loading the held-out test as the final test set. Validation acceptance was reported as 841 of 868 windows (96.8894%).

The V3 report further describes training-regime mixing as open-loop, V1 closed-loop, and V2 on-policy windows. This is a mitigation for the distribution shift that can arise when a model is trained on estimator states generated without its own corrections but later used recursively. The description is consistent with the staged windows and

on-policy configuration in the supplied materials; the original update rules and exact rollout implementation are not independently available in this workspace [8], [9]. Dataset aggregation is a relevant general reference for iterative data aggregation [5], but the chapter does not assume the project implemented every detail of that algorithm.

Unavailable training details The local files do not expose a complete training script or all experiment logs. The exact optimizer, learning rate, loss, batch size, initialization seed, hardware, and early-stopping implementation are therefore not asserted. A new researcher would need the original notebook/script, raw data and split manifest to reproduce those values and the reported checkpoint selection.

3.7 Integration of the Trained Models into the Complete System

3.7.1 GRU checkpoint loading and inference contract

GRUV3Runtime loads the checkpoint from GRU MODEL/model/PROJECT_GHOST_GRU_V3_BEST_VALIDATIO N.pt, reads input and target mean/std values from the two train-only CSV files, verifies feature/target ordering, instantiates the architecture from checkpoint configuration, and loads the saved state dictionary strictly. The runtime sets the network to evaluation mode and validates that each input is finite and has shape (101,24). It normalizes each channel, runs inference without gradient tracking, and de-normalizes the three output channels into metres per second. These safeguards establish the actual tensor contract and reduce risk of a silent feature-order mismatch.

The GRU FastAPI service exposes GET /health and POST /infer on the loopback interface in the launch configuration. The request contains a list of 101 rows of 24 values and may include the current three-axis ESKF velocity. The response includes delta_v_m_s, shape/order metadata, and optionally pseudo_velocity_m_s. The browser bridge checks service health periodically, collects samples in a rolling buffer, and transmits the most recent one-second window once its configurable 3.0-second demonstration interval elapses. The frontend does not send recorded GT or require an external network connection for local model calls.

3.7.2 Runtime correction and safety checks

The intended learned correction is a pseudo-measurement, not direct assignment to position. The browser computes or receives $v_{pseudo}=v_{ESKF}+\Delta v_{GRU}$ and submits it to ESKF.updateVelocity(). In the checked-in JavaScript bridge, a norm of the predicted Δv greater than 4.590267 m/s is rejected before the filter update; this implementation compares the magnitude of the current output with the threshold. The ESKF then evaluates the NIS for its three-dimensional innovation and rejects updates above 11.3448667301. Accepted updates use a Joseph-form covariance update. The browser service request has a finite timeout and handles unsuccessful HTTP responses without treating them as corrections.

The supplied V3 safety-calibration JSON describes a richer selected policy: a 48-dimensional mean-plus-standard-deviation summary of each normalized window scored by a train-fitted Ledoit–Wolf covariance model [6], a 95th-percentile threshold of 74.8123, a temporal gate based on the L2 difference between consecutive Δv predictions with threshold 4.590267 m/s, an NIS gate, and cooldown/recovery behavior [9]. The current JavaScript GRUBridge inspected in the repository does not implement that complete policy: its 4.590267 check is applied to the norm of the current Δv, and no OOD model or cooldown/recovery logic is present in that file. This source-versus-configuration distinction matters when interpreting both the report's validation metrics and the dashboard counters.

The DroNet service is loaded independently. Its health route reports input dimensions and whether the expected weight file exists. Its inference route accepts a 200×200 image encoded as base64 or numeric pixels, preprocesses a base64 image to grayscale and [0,1], and returns steering, collision probability, and elapsed service time. The browser obstacle module queries this endpoint and combines the response with local scan geometry and bypass decisions. The project's architecture therefore remains modular: loss of one learned branch is not equivalent to a valid output from the other branch.

Table 3.8. Runtime safety and service boundaries

Check / interface	Location and documented behavior	Interpretation
Input contract	GRUV3Runtime; 101×24 finite float window	Shape, finiteness and ordered feature names are checked.
Scaling	Train-only input and target CSV statistics	Normalization and de-normalization are performed in the Python runtime.
Current bridge pre-gate	GRUBridge.js; $\|\|\Delta v\|\| > 4.590267$ m/s	Rejects a large current output norm; not the consecutive-output metric in the calibration JSON.
Filter gate	ESKF.js; 3-DoF NIS ≤ 11.3448667	Rejects an inconsistent pseudo-velocity; accepted update uses Joseph covariance form.
Report-only additional gates	Safety-gate JSON / progress report	OOD summary distance and temporal-difference/circuit-breaker policy are not fully present in current JS bridge.
Inference APIs	GRU :8765; DroNet :8766	Separate local health and inference endpoints; no direct GRU↔DroNet model fusion.

3.8 System Execution and Operational Workflow

The repository supports a browser frontend and two local Python services. The checked-in run_simulation.bat starts the GRU and DroNet servers and then launches Vite at localhost:3000. The package scripts also expose frontend build/preview and named service commands, although the service scripts use a hard-coded GRU MODEL\.venv path. The setup below uses one fresh project-root virtual environment so both Python services can share the declared root requirements when an existing service environment points to a missing interpreter.

Table 3.9. Files needed for the principal Three.js and inference workflow

Path	Purpose
index.html; src/main.js	Browser entry and orchestration of the scene, fixed-step simulation, navigation and HUD.
src/navigation/ESKF.js; GRUBridge.js; RouteController.js	State propagation/update, GRU request lifecycle, and waypoint guidance with simulated inertial samples.
src/navigation/ObstacleAvoidanceSystem.js; src/sensors/DroneCameraSensor.js	Image acquisition, DroNet service calls, obstacle scans, and bypass/fallback logic.
GRU MODEL/model_runtime.py; model/; config/; server_fastapi.py	Frozen GRU checkpoint, normalization contract and local FastAPI inference API.
dronet_service/dronet_model.py; weights/; server.py	PULP-DroNet CNN definition, bundled weights, and local inference API.
package.json; requirements.txt; run_simulation.bat	Frontend dependencies, Python dependencies, and one-click launcher.

3.8.1 Clean setup and launch

1. Install a supported Node.js runtime (the README specifies v18 or newer) and a Python version compatible with the declared requirements. Open PowerShell in the repository root D:\G-P.
2. Create a root Python environment if the existing one is stale: `py -m venv .venv`. Install the project Python packages with `.\.venv\Scripts\python.exe -m pip install -r requirements.txt`. PyTorch installation must match the target machine; the repository declares a minimum version but does not lock a CUDA-specific wheel.
3. Install frontend packages with `npm install` when node_modules is absent or package-lock.json has changed. This installs the versions resolved by the lockfile, including Three.js and Vite.
4. Start the GRU API in a terminal from D:\G-P: `.\.venv\Scripts\python.exe -m uvicorn server_fastapi:app --app-dir "GRU MODEL" --host 127.0.0.1 --port 8765`. Confirm that the bundled checkpoint and both normalization CSV files exist under GRU MODEL before launch.
5. Start the DroNet API in another terminal: `.\.venv\Scripts\python.exe -m uvicorn server:app --app-dir dronet_service --host 127.0.0.1 --port 8766`. Confirm that `dronet_service\weights\pulp-dronet-v3-resblock-1.0.pth` is present.
6. Start the browser app in a third terminal with `npm run dev -- --host 127.0.0.1 --port 3000`, then open `http://localhost:3000`. Alternatively, use run_simulation.bat after confirming its selected Python interpreter is usable and the requirements have been installed.
7. Use the route-planning control to draw/confirm a route and start the simulation using the on-screen controls. Check the navigation HUD for GRU readiness and update/gate status, the obstacle HUD for DroNet status, and the browser console plus service terminals for request errors.
8. For a production-faithful replay, use the documented native correction cadence only after confirming the matching evaluation path. The browser demonstration cadence is 3.0 s and differs from the 0.5 s

native stride. Do not compare runs unless starting state, route, sampling, and evaluation interval are held constant.

The commands above reproduce service startup, not the historical training experiments. The npm run gru and npm run dronet entries in package.json invoke a Python executable under GRU MODEL\.venv; this may not match a newly created root `.venv`. When that nested environment is stale, start both APIs explicitly with the root interpreter as shown. The current workspace includes a stale virtual-environment reference to a removed Python installation, so environment recreation may be required. Avoid assuming that a virtual environment copied from another Python installation is portable.

3.8.2 Runtime outputs and troubleshooting

The live browser displays the reference path, hybrid estimate and estimator status, along with summary values such as current error, RMSE, peak error, drift, gate status, inference latency and call/rejection counts. The Python services expose health/inference JSON and log startup or request errors in their terminal windows. The checked-in workflow does not establish an automatic export of every trajectory sample to a CSV or a formal experiment-results database; saved screenshots should therefore be treated as snapshots rather than complete raw logs.

Table 3.10. Common launch and inference checks

Symptom	Checks grounded in this repository
GRU unavailable	Confirm port 8765 service is running; check the /health route, checkpoint path, 2 normalization CSV paths, feature order and terminal traceback.
DroNet unavailable	Confirm port 8766 and the weight file; check image endpoint format and the health response.
Browser cannot reach service	Keep service bound to 127.0.0.1, inspect browser console/CORS/network errors, and verify ports are not occupied.
Import or interpreter error	Use the active root `.venv` Python to launch Uvicorn; rebuild an environment tied to an unavailable interpreter and reinstall requirements.
No GRU update despite READY	READY indicates bridge/service availability, not accepted corrections; inspect magnitude gate, NIS, timeout and HUD counters.

Symptom	Checks grounded in this repository
Frontend command fails	Run `npm install` if dependencies are missing; use the port-3000 launch command or inspect Vite output for the selected port.

3.9 Experimental Setup and Trajectory Evaluation

Four user-supplied screenshots show live GRU V3 dashboard captures. Two capture pairs share the displayed elapsed time and reference-path length: 51.26 s with 157.6 m, and 72.05 s with 268.3 m. Within each pair, one dashboard is tagged Clear / Calm and the other Heavy Rain. Figure 3.3 reproduces the plotted areas with the screenshot's metric block transcribed beside each plot; the plot pixels are cropped only for readability. Blue denotes the displayed reference and orange the hybrid estimate according to the source legend. The source also lists a gray prediction-only ESKF line, but a separate gray curve is not clearly distinguishable in these captures.

These are 2D rendered trajectory views in the dashboard, not raw coordinate files. Their labels identify a reference and hybrid estimate, but they do not establish that the reference is a physical ground-truth flight path. The dashboard's "error", "RMSE", "peak", and "drift" values are transcribed as displayed. Since the application code that computes these screenshot metrics is not available for independent auditing, this chapter does not infer an unreported formula for its trajectory RMSE or drift value.



Figure 3.3. Four supplied live-inference snapshots. The original dashboard plots are cropped without changing their data; adjacent values and gate counters are transcribed from each capture. The source legend identifies the gray prediction-only ESKF series, although it is not independently visible in these images.

The screenshots' numeric readouts include reference and estimated XYZ coordinates, GRU Δv, gate labels, standardized-feature diagnostics, inference latency and cumulative counters. At all four captures, the number of accepted GRU updates is zero. The first pair is marked SYNTHETIC TRUST REJECT; the second pair is marked TEMPORAL JUMP REJECT. Thus the orange hybrid estimate in these images cannot be credited to an accepted GRU pseudo-velocity correction at the capture time. The supplied dashboard's trust/OOD counters and labels also cannot be fully reconstructed from the current JavaScript source, as discussed in Section 3.7.

3.10 Comparative Analysis of the Trajectory Results

Table 3.11 compares only values that are readable in the supplied captures. Within the 51.26-second pair, the clear/calm screenshot displays a lower endpoint error and displayed RMSE than the heavy-rain screenshot. The same descriptive relationship appears within the 72.05-second pair. The plotted curves are close over some portions and visibly separate in others; the size and direction of separation vary along the plotted path, so an endpoint value alone does not characterize the full curve. The clear/calm orange curve also ends earlier than the blue reference in the 157.6 m capture, while the heavy-rain estimate deviates more visibly through parts of the longer curved trajectory. These are observations about the screenshots, not proof of performance under controlled weather.

Table 3.11. Dashboard values transcribed from matched-time trajectory screenshots

Capture	Reference length	Displayed endpoint error	Displayed RMSE	Peak	Drift	Accepted GRU updates
Clear / Calm, 51.26 s	157.6 m	13.499 m	5.402 m	13.499 m	0.7480 m/s	0
Heavy Rain, 51.26 s	157.6 m	21.663 m	8.622 m	21.663 m	1.2131 m/s	0
Clear / Calm, 72.05 s	268.3 m	5.748 m	3.830 m	5.794 m	0.1400 m/s	0
Heavy Rain, 72.05 s	268.3 m	10.861 m	6.226 m	10.861 m	0.4010 m/s	0

The image “error” appears consistent with the Euclidean distance between its displayed reference and estimate coordinates: for example, the first capture’s coordinate difference is approximately 21.66 m, matching the displayed 21.663 m. This is a check on the endpoint readout only. It does not establish the implementation’s trajectory-RMSE aggregation or time weighting. To avoid misleading comparisons, reference length is kept separate from path length, deviation, endpoint distance, and reported RMSE. A shorter or visually smoother curve is not automatically a more accurate trajectory.

$$e_end = \|p_est(T) - p_ref(T)\|_2 \quad (3.5)$$

$$RMSE = \sqrt{[(1/N)\,\Sigma_i\, e_i^2]} \quad (3.6)$$

For aligned samples, e_i is the chosen error at sample i (for example, Euclidean position error). The dashboard's RMSE implementation and sample set are not available, so its displayed RMSE is reported verbatim rather than recomputed from pixels.

At the two matched time/reference-length pairs, all four captures report Accepted = 0. Consequently, the apparent lower clear/calm dashboard values cannot be attributed to GRU updates, and the pairs do not isolate the effect of weather. The comparisons may reflect a different simulation trajectory, initialization, stochastic input, or other hidden run setting. No raw coordinate series were supplied to align or recompute cross-track error, mean absolute error, traveled-path length, or samplewise trajectory RMSE.

3.11 Performance Evaluation and Discussion

The research-progress report contains a separate offline recursive-validation result for the final validation-selected Safe V3 policy [8], [9]. It reports position and velocity RMSE for three validation flights, with the baseline prediction-only ESKF compared against GRU V3. Table 3.12 transcribes those report values. The aggregate is an unweighted macro mean across the three flights, not a pooled sample-level RMSE. The reported macro position RMSE decreases from 6,421.49 m to 455.24 m (92.91% reduction), and macro velocity RMSE from 89.27 m/s to 6.98 m/s (92.18% reduction). Each of the three validation flights improves on both reported metrics under this selected policy.

Table 3.12. Reported final Safe V3 recursive validation results

Validation flight / regime tag	Position RMSE: ESKF → Safe V3	Velocity RMSE: ESKF → Safe V3	Accepted / candidate windows
indoor_1	15,015.08 → 614.10 m	169.027 → 5.951 m/s	319 / 335
wicklowTest_5 (light rain/wind tag)	2,984.59 → 394.91 m	69.882 → 6.840 m/s	290 / 290
seq10 (night tag)	1,264.80 → 356.71 m	28.897 → 8.144 m/s	232 / 243
Macro mean across 3 flights	6,421.49 → 455.24 m (−92.91%)	89.27 → 6.98 m/s (−92.18%)	841 / 868 (96.89%)

These results are substantial reductions relative to the documented baseline but do not indicate operationally accurate navigation: hundreds of metres of position RMSE remain, and velocity errors

remain several metres per second. They are validation results selected/tuned on those same three validation flights, not evidence from an independently held-out final Safe V3 test. A separately reported early closed-loop experiment (C0), which used test-flight diagnostics during development and is not a pristine final test, worsened on its three test flights: macro position RMSE increased from 8,242.51 m to 16,103.59 m and velocity RMSE from 119.09 m/s to 272.50 m/s [8]. The report therefore documents a failed early feedback configuration as well as later validation improvements; neither should be hidden when describing the development path.

Evaluation should distinguish four levels. (1) One-step residual prediction measures the network's local Δv estimate under a window distribution. (2) Recursive state-estimation evaluation measures how ESKF corrections affect accumulated position and velocity errors over an entire flight. (3) Browser dashboard snapshots describe a separate simulation-time trajectory and safety-counter state. (4) Obstacle avoidance requires separate collision or clearance metrics, which are not established by the GRU trajectory captures or by the available aggregate tables. Latency values of 12.1–26.4 ms in the four screenshots are transcribed service/dashboard values; they are not a controlled benchmark of end-to-end frame timing or embedded-hardware performance.

A sound future comparison would retain raw aligned coordinates and timestamps, evaluate all methods on identical flights and start conditions, use the same coordinate frame and time interval, and publish both per-flight and pooled metrics. Relevant measures would include position RMSE, velocity RMSE, endpoint error, cross-track deviation, correction acceptance rate, collision rate, and compute latency. Metric definitions and the chosen macro/pooling convention should be stated before comparisons are made.

3.12 Methodological Limitations and Reproducibility

3.12.1 Evidence and dataset limitations

The local project contains a frozen model package and configuration but not the full source dataset, data-loader implementation, notebook history, or every experiment log. The flight counts, row counts, split totals, model-selection narrative, and evaluation tables in Sections 3.4–3.6 and 3.11 come from the supplied 30-page progress report and integration metadata. The report describes its experiment logs as retained text summaries; the raw training archive and checkpoint were not independently replayed for this chapter. Training results should therefore be quoted as documented results rather than as a fresh reproduction.

The final Safe V3 safety policy is reported as selected on validation data, and the selected-gate metadata explicitly states that the test split was not loaded. The held-out test identities therefore do not provide a final independent performance estimate for the selected complete policy. Environmental categories are descriptive labels rather than calibrated weather measurements, and one source contributes nearly 68% of all rows. A row-level corpus total does not imply balanced representation across flights, sensor types, or environmental conditions.

3.12.2 Implementation and experiment limitations

The browser demonstration uses a 3.0-second inference cadence while the native model evaluation documents a 0.5-second correction stride. That difference changes the frequency with which corrections can affect the state. In addition, the current bridge's magnitude threshold is not the same computation as the consecutive-prediction temporal gate in the saved calibration JSON, and the JavaScript path does not show the report's OOD detector or circuit breaker. The supplied captures report OOD and synthetic-trust counters that are not fully explained by the current source. These discrepancies prevent the live snapshots from being treated as a direct reproduction of the final offline Safe V3 gate.

The browser's inertial inputs are synthesized from the simulated route/state and noise settings; the simulation does not itself validate transfer to real sensor hardware. A 3D-looking scene and trajectory visualization do not establish hardware-in-the-loop operation. The provided screenshots do not include raw point sequences or a reproducible evaluator; several displayed summaries are not sufficient to recompute samplewise metrics. The local project does not provide a controlled DroNet obstacle-avoidance benchmark or a training procedure for its bundled CNN. No MPC/NMPC or onboard timing claim is supported by the current implementation inspected here.

3.12.3 Information needed for independent reproduction

1. The exact source archives and licenses for the 19 flight records, extraction scripts, timestamp/sensor-frame manifests, and a versioned flight-identity split file.
2. The full data-preparation and training notebooks/scripts, including filtering/interpolation rules, segment exclusion records, random seeds, loss and optimizer, batch size, epoch selection, hardware/software versions, and saved per-epoch metrics.
3. The precise definition and implementation of every offline metric, including sample alignment, recursive rollout initialization, macro versus pooled aggregation, and safety-gate state transitions.
4. The exact selected checkpoint and matching gate implementation used for the reported validation results, plus a strictly untouched final test evaluation.
5. Raw time-stamped browser trajectories, deterministic simulation seeds, route definitions, weather/control settings, and the dashboard metric source used for all four screenshot captures.
6. A controlled DroNet evaluation set with obstacle labels, collision/clearance criteria, and repeatable simulation scenarios if obstacle-avoidance performance is to be quantified.

Scope This chapter describes the Three.js browser simulator and its local GRU/DroNet services. Panda3D material is intentionally excluded in accordance with the user's direction.

3.13 Chapter Summary

This chapter documented the existing browser-based navigation prototype as two connected but distinct paths: an offline recorded-flight sequence-learning workflow and a simulation-time Three.js system. The offline GRU pipeline uses flight-level splits, a one-second 100 Hz window, 24 inertial/ESKF features, train-only normalization, and the target $\Delta v = v_{GT} - v_{ESKF,pred}$. The frozen two-layer GRU produces a velocity-residual estimate that can be formed into a pseudo-velocity and applied through the 15-state ESKF's gated update. A separate pretrained PULP-DroNet model processes grayscale images for obstacle-perception logic; it is not part of the GRU residual estimator.

The report records improvements for final Safe V3 on three validation flights, but no pristine final held-out test or independent retraining is available here. The four live dashboard captures show zero accepted GRU updates, and their displayed error values therefore cannot establish that accepted GRU corrections caused the plotted behavior. The chapter also identified a difference between the selected report-time safety policy and the currently inspected JavaScript gate path. These points define the limits of the evidence and the reproducibility work still needed.

With the methodology and evidence boundaries established, the following chapter can present experimental results using the same separation between one-step residual prediction, recursive state estimation, browser simulation behavior, and obstacle-avoidance evaluation.

References

[1] J. Solà, "Quaternion kinematics for the error-state Kalman filter," arXiv:1711.02508, 2017. [Online]. Available: https://arxiv.org/abs/1711.02508.

[2] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. EMNLP, Doha, Qatar, 2014, pp. 1724–1734, doi: 10.3115/v1/D14-1179.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[4] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in Proc. IEEE/CVF CVPR, Long Beach, CA, USA, 2019, pp. 5745–5753, doi: 10.1109/CVPR.2019.00589.

[5] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in Proc. AISTATS, vol. 15, 2011, pp. 627–635.

[6] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” Journal of Multivariate Analysis, vol. 88, no. 2, pp. 365–411, 2004, doi: 10.1016/S0047-259X(03)00096-4.

[7] D. Palossi et al., “A 64-mW DNN-based visual navigation engine for autonomous nano-drones,” IEEE Internet of Things Journal, vol. 6, no. 5, pp. 8379–8395, 2019, doi: 10.1109/JIOT.2019.2905694.

[8] O. A. M. Khairi, “GPS-Free Autonomous Drone Research Progress,” supplied project progress report, 30 pp., 2026. Unpublished internal report; cited for documented dataset and experiment summaries.

[9] Project Ghost, “GRU V3 Simulation Integration Bundle,” repository files: GRU MODEL/README.md, model_runtime.py, configuration JSON/CSV files, and FastAPI service; accessed 11 Oct. 2026.

[10] Project repository, “Autonomous Drone City Simulation,” source code and package manifests in D:\G-P; accessed 11 Oct. 2026.